

LangChain & Agentic AI – From Basics to Advanced 25

RunnableParallel — Parallel Execution in LangChain

1. Quick Revision

In the previous sessions, we covered:

1. prompt | model | parser
2. invoke(), batch() and stream()
3. Sequential workflows using RunnableSequence

Main Rule

If tasks are **dependent**, that means:

Output of one task



Input of next task

then we should go for:

RunnableSequence

Example:

A



B



C

2. What If Tasks Are Independent?

Suppose our input is:

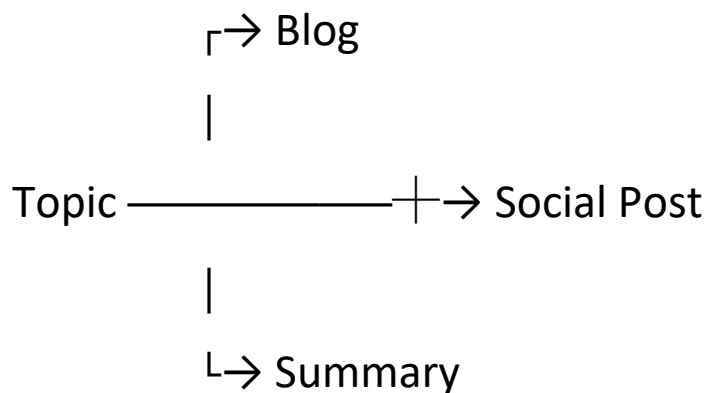
Artificial Intelligence

We need:

1. A blog
2. A social-media post
3. A short summary

These tasks are not dependent on each other.

We can think like this:



In this case, we should go for:

Parallel Execution

3. Real-Life Analogy

Friends, suppose you are organizing a birthday function.

Three people are working.

Person 1 → Orders Cake

Person 2 → Decorates Hall

Person 3 → Invites Guests

Does Person 2 have to wait until Person 1 finishes ordering the cake?

No

Can all three work independently?

Yes

That is parallel work.

Birthday Preparation

Birthday Preparation

|

|→ Cake

|

|→ Decoration

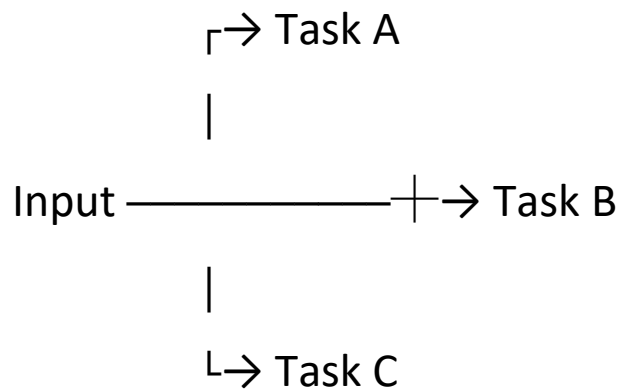
|

L→ Invitations

4. What Is Parallel Execution?

Parallel execution means multiple independent tasks can be started from the same input without waiting for one another's result.

Concept



5. Easy Memory Rule

DEPENDENCY



SEQUENCE

and:

INDEPENDENT TASKS

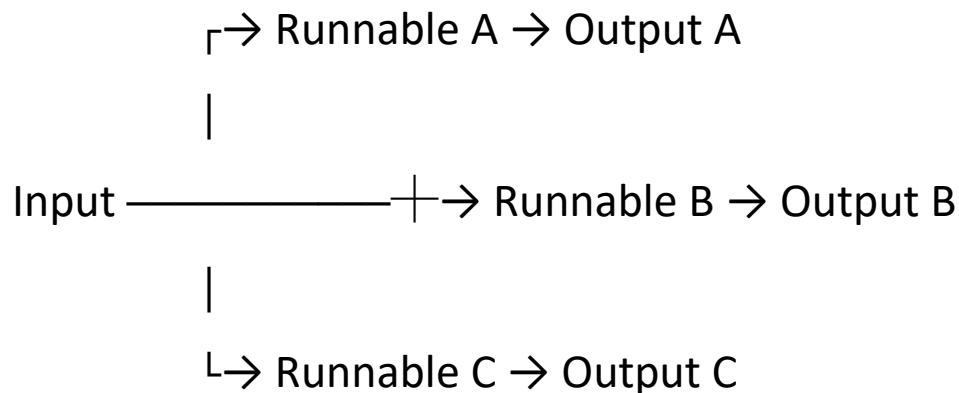


PARALLEL

6. What Is RunnableParallel?

RunnableParallel runs multiple Runnables from the same input and returns their outputs together.

Basic Architecture



These are multiple independent:

BRANCHES

Final result:

```
{  
  "A": Output_A,  
  "B": Output_B,  
  "C": Output_C  
}
```

Important Note

ONE INPUT



MULTIPLE OUTPUTS

7. Import RunnableParallel

```
from langchain_core.runnables import RunnableParallel
```

Then we can create:

```
parallel = RunnableParallel(  
    task1=...,  
    task2=...  
)
```

Each key represents one output branch.

8. First RunnableParallel Program — No AI

```
from langchain_core.runnables import (  
    RunnableLambda,  
    RunnableParallel  
)
```

```
add_ten = RunnableLambda(  
    lambda x: x + 10
```

)

```
double = RunnableLambda(  
    lambda x: x * 2
```

)

```
square = RunnableLambda(  
    lambda x: x * x
```

)

```
parallel = RunnableParallel(  
    add_ten=add_ten,  
    double=double,  
    square=square
```

)

```
result = parallel.invoke(5)
```

```
print(type(result))
```

```
print(result)
```

```
print(result["add_ten"])
```

```
print(result["double"])
```

```
print(result["square"])
```

Output

```
<class 'dict'>
```

```
{  
    'add_ten': 15,  
    'double': 10,  
    'square': 25  
}
```

```
15
```

```
10
```

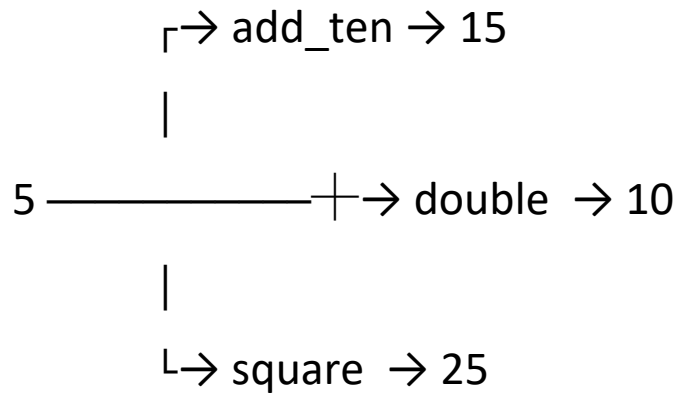
```
25
```

9. How This Program Works

Input:

5

The same input goes to every branch.



Final result:

```
{  
  "add_ten": 15,  
  "double": 10,  
  "square": 25  
}
```

Very Important

SAME INPUT



GOES TO EVERY BRANCH

10. Why Do Branch Names Matter?

Suppose:

```
parallel = RunnableParallel(  
    blog=blog_chain,  
    summary=summary_chain  
)
```

Then the final result contains:

blog

summary

We can access them easily:

```
result["blog"]
```

```
result["summary"]
```

So branch names help us identify each result.

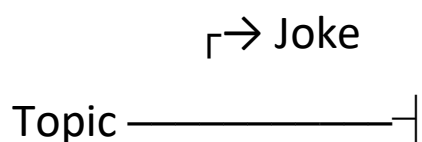
11. First AI Parallel Application

Suppose the same topic should generate:

- a joke
- a motivational message

These are independent tasks.

Architecture:



L→ Motivation

12. RunnableParallel with AI

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableParallel
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

```
joke_prompt = PromptTemplate.from_template(  
    "Tell one clean short joke about {topic}."  
)
```

```
motivation_prompt = PromptTemplate.from_template(
    """
    Give one short motivational message
    related to {topic}.
    """
)
```

```
joke_chain = (
    joke_prompt
    | model
    | parser
)
```

```
motivation_chain = (
    motivation_prompt
    | model
    | parser
)
```

```
parallel_chain = RunnableParallel(  
    joke=joke_chain,  
    motivation=motivation_chain  
)
```

```
result = parallel_chain.invoke(  
    {  
        "topic": "programming"  
    }  
)
```

```
print(result)
```

```
print("\nJOKE:")  
print(result["joke"])
```

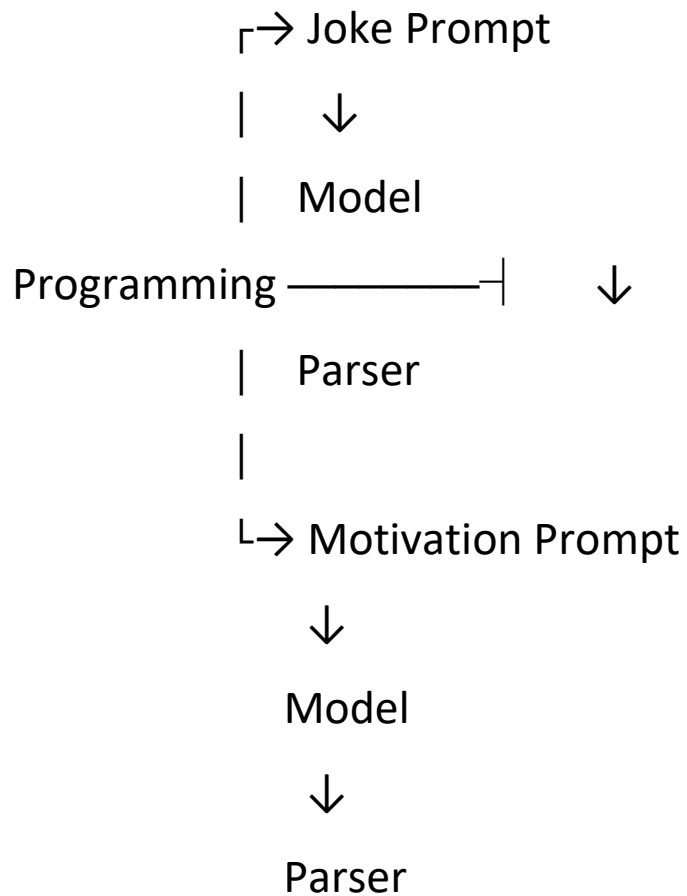
```
print("\nMOTIVATION:")  
print(result["motivation"])
```

13. Understand the Architecture

Input:

```
{  
  "topic": "programming"  
}
```

The same input goes to both branches.



Final result:

```
{  
  "joke": "...",  
  "motivation": "..."
```

}

14. Important Notes

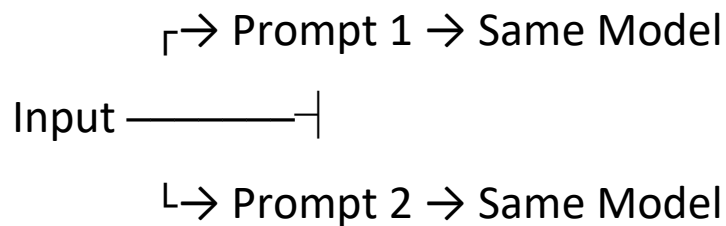
Note 1

RunnableParallel does **not** mean multiple models are mandatory.

Note 2

The same model can be used in multiple branches.

Example:



15. Practical Application — Blog + Social Post + Summary

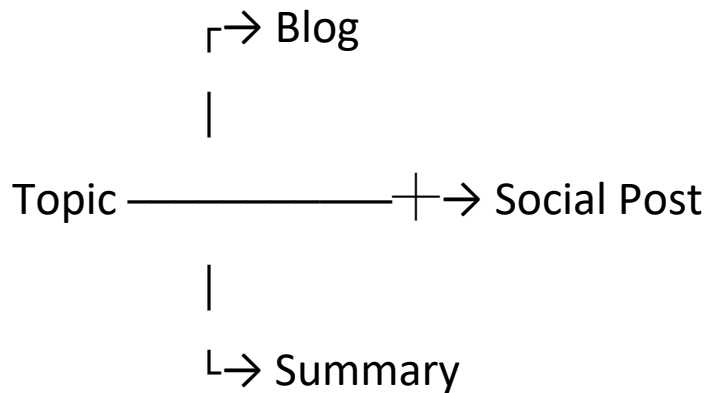
Suppose input is:

Artificial Intelligence

We need:

1. Blog
2. Social-media post
3. Short summary

Architecture:



This is a perfect use case for:

RunnableParallel

16. Create Blog Prompt

```
blog_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Write a short beginner-friendly blog
    about {topic}.
```

```
    Use simple English.
```

```
    Give a title and 5 key points.
```

```
    """
```

```
)
```

17. Create Social Prompt

```
social_prompt = PromptTemplate.from_template(
    """
    Create a short social-media post
    about {topic}.

    Keep it engaging and simple.
    """
)
```

18. Create Summary Prompt

```
summary_prompt = PromptTemplate.from_template(
    """
    Give a 3-line summary of {topic}
    for a beginner.
    """
)
```

19. Create Individual Chains

```
blog_chain = (
```

```
blog_prompt
| model
| parser
)
```

```
social_chain = (
    social_prompt
    | model
    | parser
)
```

```
summary_chain = (
    summary_prompt
    | model
    | parser
)
```

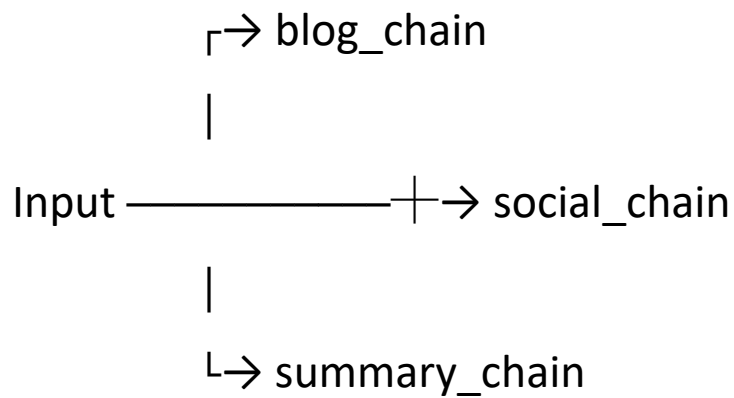
These are three independent chains.

20. Combine Them with RunnableParallel

```
parallel_chain = RunnableParallel(
```

```
    blog=blog_chain,  
    social_post=social_chain,  
    summary=summary_chain  
)
```

Concept:



21. Execute

```
result = parallel_chain.invoke(  
    {  
        "topic": "Artificial Intelligence"  
    }  
)
```

All branches receive:

```
{  
    "topic": "Artificial Intelligence"
```

```
}
```

22. Complete Blog + Social Post + Summary Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableParallel
```

```
model = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
parser = StrOutputParser()
```

```
blog_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Write a short beginner-friendly blog
```

about {topic}.

Requirements:

- Simple title
- Simple introduction
- 5 important points
- One-line conclusion

"""

)

social_prompt = PromptTemplate.from_template(

"""

Create a short social-media post
about {topic}.

Keep it simple and engaging.

"""

)

summary_prompt = PromptTemplate.from_template(

```
"""
```

```
Summarize {topic} in 3 short lines  
for a beginner.
```

```
"""
```

```
)
```

```
blog_chain = (  
    blog_prompt  
    | model  
    | parser  
)
```

```
social_chain = (  
    social_prompt  
    | model  
    | parser  
)
```

```
summary_chain = (  
    summary_prompt
```

```
    | model
    | parser
)
```

```
parallel_chain = RunnableParallel(
    blog=blog_chain,
    social_post=social_chain,
    summary=summary_chain
)
```

```
topic = input("Enter topic: ")
```

```
result = parallel_chain.invoke(
    {
        "topic": topic
    }
)
```

```
print("\n=====")
print("BLOG")
```

```
print("=====")
```

```
print(result["blog"])
```

```
print("\n=====")
```

```
print("SOCIAL POST")
```

```
print("=====")
```

```
print(result["social_post"])
```

```
print("\n=====")
```

```
print("SUMMARY")
```

```
print("=====")
```

```
print(result["summary"])
```

23. Execution Flow

└→ Blog Prompt

| ↓

| Model

| ↓

| Parser

| ↓

| Blog

|

|→ Social Prompt

Topic ————— | ↓

| Model

| ↓

| Parser

| ↓

| Social Post

|

|→ Summary Prompt

↓

Model

↓

Parser



Summary

Final result:

```
{  
  "blog": "...",  
  "social_post": "...",  
  "summary": "..."  
}
```

24. RunnableSequence vs RunnableParallel

RunnableSequence

Tasks are dependent

One after another

Previous output becomes
next input

Order matters

One main flow

$A \rightarrow B \rightarrow C$

RunnableParallel

Tasks are independent

Multiple branches

Same input goes to branches

Dependency between branches is
not required

Multiple outputs

$A \rightarrow B, C, D$

25. Easy Diagram

RunnableSequence

Input



Task A



Task B



Task C



Output

RunnableParallel

└→ Task A → Output A



Input ———┴→ Task B → Output B



└→ Task C → Output C

26. Can RunnableSequence and RunnableParallel Be Combined?

Yes.

Very very important.

Real applications may need:

Sequence

+

Parallel

27. Example Requirement

Step 1

Generate an explanation.

Then:

Step 2

From that explanation, independently create:

- summary
- quiz
- social post

Architecture:

Topic



Generate Explanation

↓

Explanation

↓

$\Gamma \rightarrow$ Summary

|

$\vdash \rightarrow$ Quiz

|

$\vdash \rightarrow$ Social Post

This combines:

RunnableSequence

+

RunnableParallel

28. Complete Sequence + Parallel Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import (
```

```
RunnableLambda,  
RunnableParallel  
)
```

```
model = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
parser = StrOutputParser()
```

Step 1

```
explanation_prompt = PromptTemplate.from_template(  
    """"
```

```
    Explain {topic} in very simple English.
```

```
    Give:
```

- Definition
- 5 key points

- One example

```
""""
```

```
)
```

```
explanation_chain = (  
    explanation_prompt  
    | model  
    | parser  
)
```

```
# Convert string to dictionary  
prepare_input = RunnableLambda(  
    lambda text: {  
        "content": text  
    }  
)
```

```
# Parallel Branch 1
```

```
summary_prompt = PromptTemplate.from_template(
    """
    Summarize the following content
    in 3 short lines:

    {content}
    """
)
```

```
summary_chain = (
    summary_prompt
    | model
    | parser
)
```

Parallel Branch 2

```
quiz_prompt = PromptTemplate.from_template(
    """
    Generate 5 beginner MCQs
```

from the following content:

```
{content}
```

```
"""
```

```
)
```

```
quiz_chain = (
```

```
    quiz_prompt
```

```
    | model
```

```
    | parser
```

```
)
```

```
# Parallel Branch 3
```

```
social_prompt = PromptTemplate.from_template(
```

```
    """
```

```
Create a short social-media post
```

```
from the following content:
```

```
{content}
```

```
""""  
)
```

```
social_chain = (  
    social_prompt  
    | model  
    | parser  
)
```

```
parallel = RunnableParallel(  
    summary=summary_chain,  
    quiz=quiz_chain,  
    social_post=social_chain  
)
```

```
final_chain = (  
    explanation_chain  
    | prepare_input
```

```
    | parallel  
)
```

```
result = final_chain.invoke(  
    {  
        "topic": "Machine Learning"  
    }  
)
```

```
print("\nSUMMARY")  
print(result["summary"])
```

```
print("\nQUIZ")  
print(result["quiz"])
```

```
print("\nSOCIAL POST")
```

```
print(result["social_post"])
```

29. Understand the Complete Architecture

```
{"topic": "Machine Learning"}
```



Explanation Prompt



Model



Parser



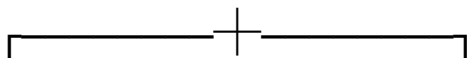
Explanation String

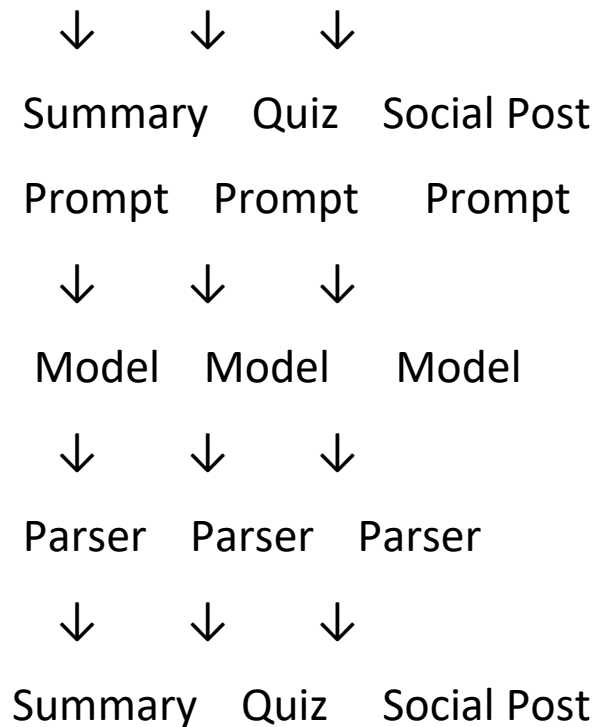


RunnableLambda



```
{"content": explanation}
```





30. Why Do We Need RunnableLambda Here?

After:

`explanation_chain`

we get a string.

Example:

Machine Learning is...

But the parallel branch prompts expect:

`{content}`

Therefore, we convert:

String

```
into:
{
    "content": "...
}
using:
prepare_input = RunnableLambda(
    lambda text: {
        "content": text
    }
)
```

So the flow becomes:

Explanation String



RunnableLambda



Content Dictionary



RunnableParallel

31. What Does Each Parallel Branch Receive?

The parallel component receives:

```
{  
    "content": explanation  
}
```

The same input goes to:

summary_chain

quiz_chain

social_chain

So:

$\Gamma \rightarrow$ Summary

Explanation Dictionary $\dashv \rightarrow$ Quiz

$\mathbb{L} \rightarrow$ Social Post

32. Important Points

1. Parallel execution is used for independent tasks.
2. RunnableParallel runs multiple Runnables from the same input.
3. Same input is sent to every branch.
4. Each branch has a name.

Example:

```
RunnableParallel(  
    joke=joke_chain,  
    motivation=motivation_chain  
)
```

5. The final result is usually dictionary-like.

Example:

```
{  
    "joke": "...",  
    "motivation": "..."  
}
```

6. We can access an individual result using its key.

```
result["joke"]
```

7. RunnableParallel does not require multiple models.

8. The same model can be reused in multiple branches.

9. RunnableSequence is used when tasks are dependent.

10. RunnableParallel is used when tasks are independent.

11. Sequence and Parallel can also be combined.

33. Master Decision Rule

Ask one simple question:

Does the next task need the output of the previous task?

If YES

DEPENDENCY



RunnableSequence

Example:

Generate Outline



Write Blog Using Outline

If NO

INDEPENDENT



RunnableParallel

Example:

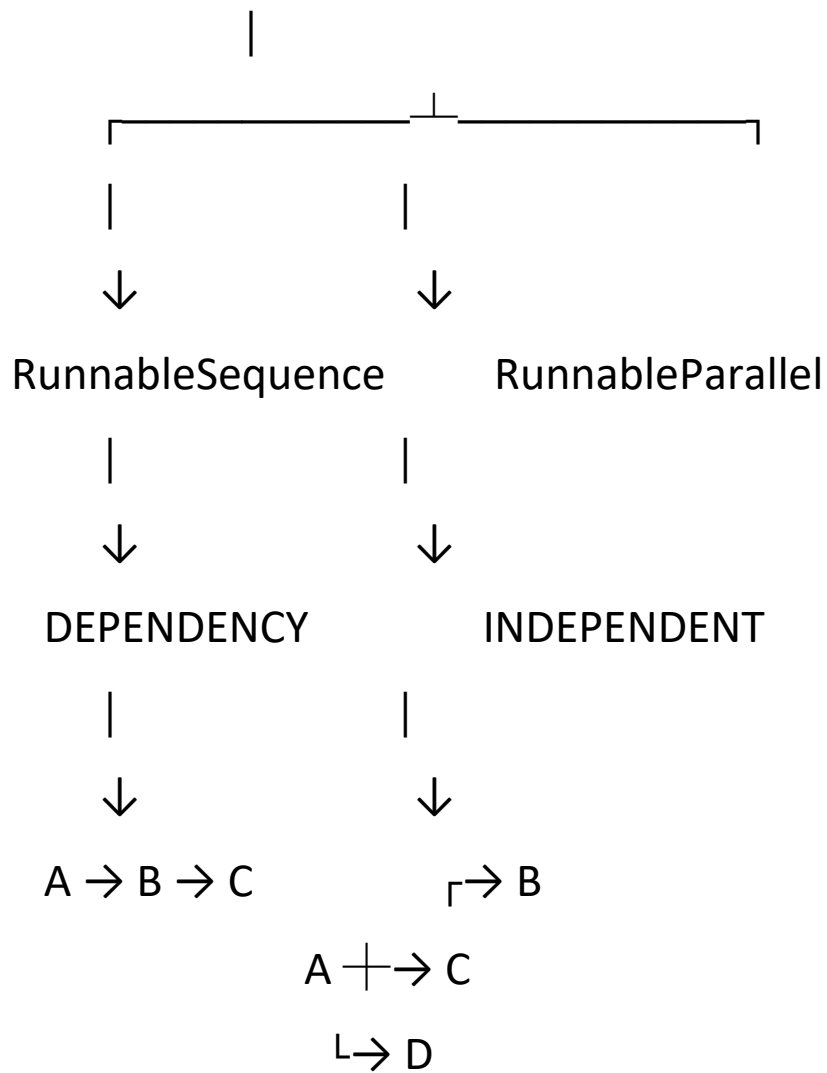
└→ Blog

Topic —————┴→ Tweet

└→ Summary

34. Master Diagram

RUNNABLE WORKFLOWS



35. Sequence + Parallel

Real applications may use both.

Input



Task A

↓

Task B

↓

$\vdash \rightarrow$ Task C

$\vdash \rightarrow$ Task D

$\vdash \rightarrow$ Task E

Here:

Input $\rightarrow$ A $\rightarrow$ B

is sequential.

Then:

$\vdash \rightarrow$ C

B $\xrightarrow{\vdash}$ D

$\vdash \rightarrow$ E

is parallel.

36. Final Memory Formula

RunnableSequence

DEPENDENCY

A

↓

B

↓

C

RunnableParallel

NO DEPENDENCY

$\Gamma \rightarrow B$

$A \dashv \vdash \rightarrow C$

$L \rightarrow D$

Combined Workflow

A

↓

B

↓

$\Gamma \rightarrow C$

$\vdash \rightarrow D$

$L \rightarrow E$

One-Line Conclusion

Use RunnableSequence when one task depends on the previous task. Use RunnableParallel when multiple independent tasks can work from the same input. Real-world applications can combine both Sequence and Parallel workflows.